

1. What is a CSP?

- A constraint satisfaction problem (CSP) consists of variables, domains, and constraints.
- The goal is to assign a value to each variable so that all constraints are satisfied.
- CSPs are useful for many combinatorial problems in AI.

Key Idea: A CSP solves a problem by finding a complete and consistent assignment of values to variables.

Example: Map coloring, n-queens, scheduling, Sudoku, etc.

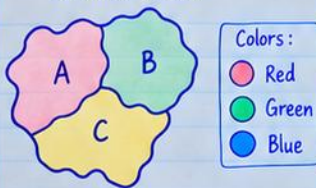
3. Types of Constraints

- Unary constraints:** Restrict the value of a single variable.
e.g., $X \neq \text{red}$.
- Binary constraints:** Restrict the values of two variables.
e.g., $X \neq Y$.
- Higher-order constraints:** Involve three or more variables.
e.g., $X + Y + Z = 10$.
- Hard constraints:** Must always be satisfied.
- Soft constraints (preferences):** Desirable but not required; often used in optimization problems.

5. Classic Examples

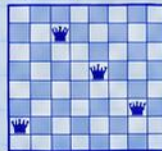
(a) Map Coloring

- Variables: regions on a map.
- Domains: colors (e.g., {red, green, blue}).
- Constraints: adjacent regions must have different colors.



(b) N-Queens

- Variables: one per column (or per row).
- Domains: row positions $\{1, 2, \dots, N\}$.
- Constraints: no two queens can attack each other (same row, column, or diagonal).



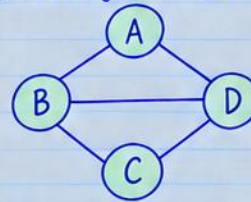
2. Core Terms

- Variable:** A symbol that must be assigned a value.
- Domain:** The set of possible values for a variable.
- Constraint:** A restriction on the allowed combinations of values for a set of variables.
- Assignment:** A mapping from variables to values.
- Complete assignment:** Assigns a value to every variable.
- Partial assignment:** Assigns values to some variables.
- Consistent assignment:** An assignment that does not violate any constraint.

Variable : X (e.g., a region, queen, or time slot)
 Domain : $D(X) = \{\text{possible values}\}$
 Constraint : A relation limiting allowed combinations
 Goal : Find a complete and consistent assignment

4. Constraint Graph

- A constraint graph has a node for each variable.
- An edge between two nodes indicates a constraint between the corresponding variables.
- The graph structure can simplify reasoning and help choose good variable orderings.



Example:

Variables: $\{A, B, C, D\}$
 Edges show constraints (e.g., $A \neq B$, $A \neq D$, $B \neq C$, $C \neq D$, $B \neq D$).

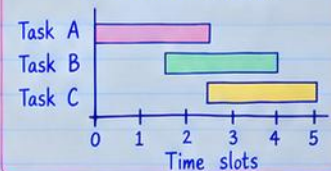
(c) Sudoku

- Variables: each cell in the grid (e.g., 9×9).
- Domains: digits $\{1, 2, \dots, 9\}$.
- Constraints: each row, column, and 3×3 block must contain all digits exactly once.

5	3		7	
6		1	9	5
	9	8		

(d) Scheduling

- Variables: tasks or activities.
- Domains: possible time slots.
- Constraints: tasks that conflict cannot occur at the same time (e.g., share a resource or have a precedence relation).



6. Why CSPs Matter

- Provide a compact and natural representation for many problems.
- Exploit problem structure (constraints and graph structure).
- Allow specialized algorithms and heuristics (e.g., variable ordering, constraint propagation).
- Often solve problems much more efficiently than ordinary search through all possible states.

7. Key Takeaways

- A CSP consists of variables, domains, and constraints.
- The goal is to find a complete and consistent assignment.
- Constraints can be unary, binary, or higher-order, and can be hard or soft.
- The constraint graph represents the variable-constraint structure.
- CSPs arise in many real-world problems (map coloring, n-queens, Sudoku, scheduling, etc.).
- CSPs are important because they provide a compact representation and can often be solved much more efficiently than ordinary search.

1. Backtracking Search

- Depth-first search over variable assignments.
- Assign one variable at a time.
- If a constraint is violated, backtrack to try a different value.
- Complete for finite CSPs (will find a solution if one exists).

Key Idea: Build a partial assignment, extending it one variable at a time, and backtrack when it becomes inconsistent.

Example: Map coloring — assign a color to each region such that adjacent regions have different colors.

3. Variable Ordering Heuristics

- Choose the next variable to assign using a heuristic.
- Helps reduce the size of the search tree.
- Two common heuristics:

(a) Minimum Remaining Values (MRV)

- Choose the variable with the fewest legal values remaining.
- Also called the “first-fail” heuristic.
- Idea: variable most likely to cause a failure, so deal with it early.

Example:

If $|\text{Domain}(X)| = 2$, $|\text{Domain}(Y)| = 4$, $|\text{Domain}(Z)| = 3$, choose X (MRV).

(b) Degree Heuristic

- Break ties among MRV variables using the degree heuristic.
- Choose the variable involved in the largest number of constraints with other unassigned variables.
- Idea: assigning this variable is likely to reduce the search space more.

Example: If X and Y both have 2 values left, but X is constrained with 4 neighbors and Y with 2 neighbors, choose X.

6. Search Example (Map Coloring)

Australia map coloring problem

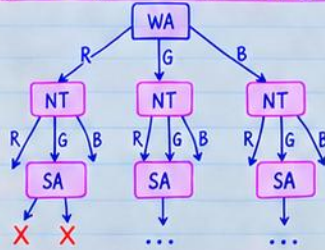


Variables: {WA, NT, SA, Q, NSW, V, T}

Domains: {R, G, B}

Constraints: Adjacent regions must have different colors.

Partial assignment tree (backtracking)



If a constraint is violated, backtrack and try the next value.

2. Basic Algorithm

Backtracking-Search (CSP)

- Choose an unassigned variable X.
- Choose a value v from the domain of X.
- Test if assigning $X = v$ is consistent (does not violate any constraints).
- If consistent, recursively assign the next variable.
- If failure occurs (no valid value), backtrack and try a different value.

Pseudocode (high level)

```
function BACKTRACKING-SEARCH (CSP)
  return RECURSIVE-BACKTRACKING({})

function RECURSIVE-BACKTRACKING (assignment)
  if assignment is complete
    return assignment
  X ← SELECT-UNASSIGNED-VARIABLE()
  for each value v in ORDER-VALUES(X)
    if CONSISTENT(X, v, assignment)
      add {X = v} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment)
      if result ≠ failure return result
    remove {X = v} from assignment
  return failure
```

4. Value Ordering

Least Constraining Value (LCV)

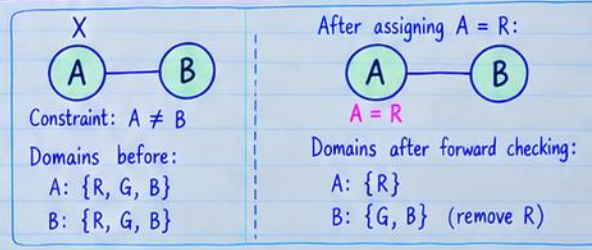
- Choose the value that rules out the fewest options for the neighboring variables.
- Leaves the maximum flexibility for future assignments.
- Helps avoid dead ends further down the tree.

Example: For a variable X, evaluate each possible value by counting how many values it eliminates from the domains of neighboring variables, and choose the value with the smallest count.

5. Inference During Search

- Forward checking: after assigning a value to a variable, remove inconsistent values from the domains of its unassigned neighbors.
- Detects future conflicts early.
- Prunes values that cannot possibly be part of a solution.

Example: Domain reduction



7. Why Heuristics Help

- Reduce the branching factor (fewer choices to try).
- Find dead ends earlier in the search.
- Guide the search toward promising states.
- Can speed up search dramatically, making large CSPs solvable.

8. Key Takeaways

- Backtracking search assigns one variable at a time and backtracks on constraint violations.
- Use MRV and degree heuristics to choose the next variable.
- Use LCV to choose the best value for a variable.
- Forward checking prunes inconsistent values early.
- Heuristics reduce the search space and find dead ends sooner.
- Backtracking search is complete for finite CSPs.

1. Forward Checking vs Constraint Propagation

- Forward checking (FC) prunes values that are immediately inconsistent with the current assignment (looks only one step ahead).
- Constraint propagation goes further by repeatedly enforcing local consistency, pruning additional values that become inconsistent due to earlier pruning.

Key Idea: Constraint propagation looks beyond immediate conflicts and uses the constraints to infer more domain reductions.

2. Arc Consistency

- A binary constraint $X - Y$ is arc consistent if for every value x in the domain of X , there exists some value y in the domain of Y that is compatible with the constraint.
- If not, remove x from the domain of X .
- Arc consistency is usually defined for binary constraints.

Definition (Arc $X \rightarrow Y$):

$X \rightarrow Y$ is arc consistent if for every $x \in D(X)$, there exists a $y \in D(Y)$ such that (x, y) satisfies the constraint between X and Y .

3. AC-3 Algorithm

Algorithm (AC-3)

- Initialize a queue with all arcs (X, Y) in the CSP.
- Remove an arc (X, Y) from the queue.
- Revise the domain of X by removing values that have no compatible value in Y .
- If domain of X shrinks, add all neighboring arcs (Z, X) for $Z \neq Y$ back to the queue.
- Repeat until the queue is empty or a domain becomes empty (failure).

Revise(X, Y):

- For each value x in $D(X)$, check if there exists a value y in $D(Y)$ such that (x, y) is consistent.
- If no such y exists, remove x from $D(X)$.
- Return true if any value was removed.

4. Example: Domain Pruning with Arc Consistency

Variables: X, Y

Domains: $D(X) = \{1, 2, 3\}$

$D(Y) = \{2, 3\}$

Constraint: $X < Y$

Apply arc consistency:

- Revise(X, Y):**
 - $x = 1$ has a support ($y = 2$ or 3)
 - $x = 2$ has a support ($y = 3$)
 - $x = 3$ has no support (no $y > 3$) $\rightarrow$ remove 3 from $D(X)$
- Revise(Y, X):**
 - $y = 2$ has a support ($x = 1$)
 - $y = 3$ has a support ($x = 1$ or 2) $\rightarrow$ no change to $D(Y)$

Domains before:

$D(X) = \{1, 2, 3\}$

$D(Y) = \{2, 3\}$



Domains after AC-3:

$D(X) = \{1, 2\}$

$D(Y) = \{2, 3\}$

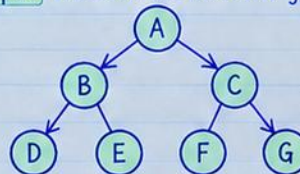
5. Other Consistency Ideas

- Path consistency:** For any two variables X and Z , for every compatible pair (x, z) , there must exist a value y for any intermediate variable Y such that (x, y) and (y, z) are both consistent.
- k-consistency:** A generalization where any consistent assignment to $k - 1$ variables can be extended to a consistent assignment to a k th variable.
- Stronger forms of consistency (e.g., path consistency, k-consistency) provide more pruning but require more computation.

6. Tree-Structured CSPs

- If the constraint graph is a tree, CSPs can be solved efficiently in linear time.
- Use a root ordering and perform:
 - A backward pass to compute consistent domains (from leaves to root).
 - A forward pass to assign values (from root to leaves).
- Each variable has at most one parent (except the root), so local consistency guarantees a global solution.

Example: Tree-structured constraint graph



Key Idea:

On a tree, enforce arc consistency in a rooted order, then assign values from root to leaves. The result is a complete and consistent solution.

7. Benefits and Costs

Method	Pruning Power	Computational Overhead
Forward Checking (FC)	Prunes only immediate conflicts (one step ahead).	Low (simple and fast at each node).
Arc Consistency (AC-3)	Stronger pruning by enforcing local consistency (repeatedly).	Higher (maintains a queue and revises multiple arcs, but still polynomial time).

8. Key Takeaways

- Forward checking prunes only immediate conflicts.
- Constraint propagation (e.g., arc consistency) repeatedly enforces local consistency.
- Arc consistency: every value must have a compatible value in neighboring domains.
- AC-3 maintains a queue of arcs and revises domains.
- Stronger consistency (path, k-consistency) gives more pruning but costs more computation.
- Tree-structured CSPs can be solved efficiently with a backward and forward pass.

1. Local Search for CSPs

- Another approach is local search, using a complete-state formulation.
- Start with a complete assignment (may violate constraints).
- Use the min-conflicts heuristic: repeatedly repair conflicted variables.
- Very effective for large CSPs such as the N-Queens problem.

Key Idea: Min-conflicts tries to minimize the number of violated constraints by changing the value of one variable at a time.

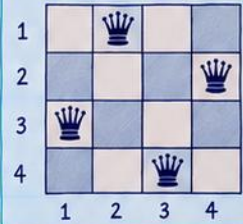
2. Min-Conflicts Example

- Start with a complete assignment (e.g., random).
- Choose a conflicted variable (one that violates at least one constraint).
- Assign the value that minimizes the number of violated constraints (break ties randomly).
- Repeat until no conflicts remain (solution found) or a step limit is reached.

Example: N-Queens problem (N=4)

- Place 4 queens on a 4x4 board so that no two queens attack each other.
- Variables: columns for each row.
- Use min-conflicts to reduce the number of attacking pairs.

A solution for 4-Queens:



3. Structure Matters

- Many CSPs have independent subproblems that can be solved separately.
- Cutset conditioning: choose a small set of variables (cutset), try all assignments, and solve the remaining independent subproblems.
- Exploiting sparse constraint graphs can make solving easier.
- Decomposition of large problems can greatly simplify CSPs by breaking them into smaller, loosely connected parts.

4. Soft Constraints and Preferences

- In some problems, constraints can be violated at a cost (rather than being strictly forbidden).
- The optimization version seeks the best assignment that minimizes the total cost (or maximizes preference).
- Allows trade-offs between conflicting objectives.
- Examples: scheduling (soft deadlines), timetabling (preferred times), and resource allocation with costs.

5. Real-World Applications

- Scheduling: assign tasks to times or resources.
- Register allocation: assign variables to machine registers.
- Timetabling: schedule classes, exams, or meetings.
- Resource assignment: allocate limited resources efficiently.
- Puzzles: Sudoku, map coloring, N-Queens.
- Vision: labeling and interpretation problems (e.g., image segmentation).

6. CSP Strategy Summary

Algorithm / Technique	Main Idea	When It Is Useful
Backtracking + heuristics (MRV, LCV, etc.)	• Systematic search with heuristics. • Use MRV to select variable, LCV to order values.	Good general-purpose method. Effective on many structured CSPs.
Forward Checking	• After assigning a variable, remove inconsistent values from neighboring variables.	Reduces search by early detection of failures. Useful for medium-sized problems.
Arc Consistency (AC-3)	• Enforce arc consistency by removing inconsistent values from all variable domains. • Can be used as preprocessing or during search.	Good for tighter constraint propagation. Useful when constraints are strong and domains are not too large.
Min-Conflicts (local search)	• Start with a complete assignment. • Repeatedly repair a conflicted variable (minimize conflicts).	Very effective for large, loosely constrained problems (e.g., N-Queens, large timetabling).

7. Common Exam Points

- Define a Constraint Satisfaction Problem (CSP).
- Explain the MRV heuristic and LCV heuristic.
- Describe forward checking and how it works.
- Explain arc consistency and the AC-3 algorithm.
- Describe the min-conflicts heuristic.
- Compare systematic search (backtracking) and local search approaches for CSPs.

8. Chapter 5 Key Takeaways

1. A CSP consists of variables, domains, and constraints.
2. Backtracking search with heuristics (MRV, LCV) can greatly reduce search effort.
3. Forward checking and arc consistency prune inconsistent values.
4. Local search (min-conflicts) is simple and very effective for large CSPs such as N-Queens.
5. Exploiting problem structure (independence, cutsets, sparse graphs) can simplify CSPs.
6. Some problems allow soft constraints with costs, solved as optimization.
7. CSP techniques have many real-world applications.